

Password-based Systems Best Practices

Nowadays data represents one of the most valuable assets in almost any field and for almost any company. That's why there is interest in "stealing" it through cyber attacks and, of course, interest in defending it.

During the years, techniques for both attacking and defending the data became more and more advanced. Standards for making safe systems have been created, and research about these themes is constantly going on. Also, guidelines for information systems users are updated: passwords are the "first line" of defense, and for sure the first thing an attacker looks at, this is why there is very much attention on them.

Even though the great effort in making good guidelines for passwords, every year there are thousands of cyber attacks based on passwords weaknesses.

To understand how this is possible, we'll go through their life-cycle - from the creation, to the management, the usage and the compromise - and discuss the best practices for password-based systems.

Entropy is a measure of unpredictability: a good password has high entropy. A user with no constraints would choose passwords that are easy to remember: simple and with low entropy, so predictable. That's why password systems force users to use capital letters, digits and special characters too. Even though this measure, human generated passwords often have low entropy: common words are more likely to be used than random sequences of characters. Also, usernames and passwords are often reused, making it for an attacker to have access to multiple accounts with a single user-pwd pair.

A 'credential stuffing attack' consists in using lists of compromised user credentials to breach into a system, based on the assumption that many users recycle them.

A 'dictionary attack' is a brute force attack in which common words are used as possible passwords (maybe combined with digits and some special characters (even among special characters there are most common ones (even this fact lowers down human generated passwords' entropy))).

Both of these two attacks are based on automation and on the assumption that the user doesn't make a great effort in choosing passwords. That's why automation can come in help also for password managing.

The goal is to make the intruder job harder: finding an unknown password should require a lot of time. Hashes and salts are a beginning of solution. Hashing a passwords means transforming it into a different sequence, from which it's impossible to reconstruct the original one. This implies that it's not the password to be saved in servers, but its hash: breaking into a server gives the attacker a list of hashes, not of passwords. And to log in as a user, finding a correct password is still needed. Of course, having a list of hashes is an advantage: it allows the comparison of a guessing with more than one password at the same time. Salt is a technique used to make the finding of a correct password even harder: it's appended to the password before the hashing, so that a guessing has to be hashed several times (with different salts) in order to be compared with the breached hashes.

A counter measure against the hash technique is represented by the rainbow table attack: it consists of making chains of hashes of potential passwords, allowing the attacker to efficiently reverse a hash.

Once again, the goal of who attacks is to save time. So, the goal of who defends is the opposite. And a way to make times longer is the key stretching: a password is not hashed once, but twice or more. And being this a sequential computation, it makes the usage of GPU useless for trying to save time. An important aspect of cyber security is the tradeoff between 'safe' and 'usable': if we think about making systems with a high number of steps in key hashing, we make the access time higher than what the user considers acceptable.

The user experience is one of the most important aspects in security: the A in the AIC (?) triad is fundamental, and user waiting time is part of it.

And another important thing related to users is about password complexity: asking the user to make more and more complex passwords revealed to be ineffective, sometimes counterproductive - as shown in pictures of stickers with passwords next to screens.

This is why the state of the art password management includes two things. Password managers to help the user and security measures that go beyond the password usage: MFA.

The first one makes possible to use really random passwords. The second one makes the password breaching only the first step for an attacker.

Before concluding our exploration, it's important to remember that when designing an information system and its security measures, we have to take decisions about money; how much can we spend, how we want to spend and - most importantly - how much do we think that possible attackers would spend to attack us? How much we want to spend is not only about money but also usability: the safest system is disconnected from the internet, but it's useless.

Anyway, as of today, there are services that tell us when some password has been breached: we can think about prevention but also be ready to repair when something bad happens.

As a conclusion, we can say that password security is not only a technic problem: it's about user education, password life-cycle management, automatic tools and repairing strategies. And of course, as always in this field, about threat modeling.

Sometimes the user is the only weak point but we have to remember something important: with no people there would be no problem; but there would be no need for cyber security; so, there would be no such field.